

Educational AI Guardrail: Safe RAG Pipelines for University Regulations

By Maedeh Amini

1. Why I Chose This Domain

I chose the education domain because I lived this problem myself. When I started my Cognitive Science studies at Osnabrück University as an international student, the German educational system and rules were completely new to me. On top of that, our study program regulations change multiple times.

It was a huge burden to search for the right information. Mostly, I had to contact the examination office or the study planning consultant, but because of their high workload, they replied to emails quite late, leaving us on a long waiting list.

Also, even though my program is taught entirely in English, about 90% of the actual regulation documents are written in German. At the time, my German was at a B1 level, and understanding legal laws requires at least a C1 level. I felt there should be a reliable AI agent to help students with these inquiries. But for something as important as study regulations, the AI absolutely cannot make mistakes.

2. The Failures I Wanted to Prevent

For a university chatbot, getting the facts wrong can ruin a student's study plan. I focused on fixing three main issues:

1. **Making things up (Hallucinations):** The model giving incorrect advice about degree rules but sounding super confident about it.
2. **Getting lost (Off-topic drift):** The model losing track of the conversation when a student asks a complicated, multi-part question.
3. **Language mix-ups (Format issues):** The model failing to search German documents properly when a student asks a question in English.

3. How I Built It (My Architecture)

To solve these problems, I set up two different systems to see which one was safer: a standard **Semantic RAG** and a **Knowledge Graph RAG (GraphRAG)**.

Input-Side Guardrails

- **Handling messy files:** University PDFs are full of tables and weird formatting. I used RAGFlow (specifically its `DeepDoc` parser) because it's really good at breaking down complex files, images, and tables properly.
- **Fixing the language gap:** To make sure English questions could successfully find German answers, I used a multilingual embedding model (`BAAI/bge-m3`).
- **Connecting the dots:** Standard RAG systems struggle to remember things over long texts. To fix this, I used Neo4j to build a Knowledge Graph. This acts as a guardrail because it maps out exactly how different rules connect to each other, which keeps the AI on track.

Output-Side Guardrails

Instead of just hoping the answers were right, I used automated tools like BERTScore and LLM-as-a-judge (DeepEval framework) to grade the AI's responses. When selecting an LLM-as-a-judge framework, both RAGAS and DeepEval were considered. RAGAS was ultimately discarded due to insufficient documentation, which hindered the resolution of execution errors during initial testing. Instead, DeepEval was adopted because of its comprehensive documentation and established presence in the literature. To rigorously evaluate the RAG architectures, Contextual Recall and Contextual Precision were employed to assess the retrieval mechanisms. Furthermore, the generation mechanisms were evaluated using Answer Relevancy and Faithfulness, with the latter utilized specifically to detect and measure model hallucinations.

I specifically measured:

- Did it actually find the right information?
- Is the final answer strictly based on the documents, or did it hallucinate?

4. Frameworks & Tools

The prompt mentioned using what excites us, so I mixed a few different approaches to build the safest pipeline possible:

- **Orchestration:** I used `LangChain` to tie everything together. It made it easy to set up my standard RAG baseline using their recursive character text splitter for benchmark data.
- **Data Parsing:** I used RAGFlow. Their OCR and `DeepDoc` parser are great for dealing with the complex formatting, tables, and images you usually find in academic PDFs.
- **Databases:**
 - For the standard Semantic RAG, I used FAISS because it's lightweight and builds the vector index incredibly fast.

- For the GraphRAG, I used Neo4j. I specifically used the `SimpleKGPipeline` from the `neo4j_graphrag` library. I enabled it to handle both semantic similarity checks and graph traversal via Cypher queries.
- **Evaluation (LLM-as-a-Judge):** DeepEval and BERTScore. As I mentioned above, DeepEval gave me much better control and better documentation than RAGAS for scoring faithfulness and relevance.
- **Embeddings:** BAAI/bge-m3. A 1024-dimensional multilingual model that literally saved the project from failing on German documents than the smaller 384-dimensional unilingual model (`all-MiniLM-L6-v2`) .

5. Trade-Offs I Made

- **Safety over Speed:** I chose safety. GraphRAG is a bit slower because searching through a graph takes more time, but it's much better at preventing made-up answers.
- **Privacy over Easy APIs:** I couldn't just send all this university data to OpenAI or Claude because of strict European privacy laws. To keep the data safe, I used an open-source model (`gemma-4-31B-it-FP8-dynamic`) and ran it on local servers.
- **Coverage over Simplicity:** The standard Semantic RAG is much easier to set up, but it drops the ball on complicated questions. The GraphRAG was harder to build, but it covers more ground because it can find the most accurate response across scattered pieces of information.

❖ System Evaluation & Reflection

1. How I Know It Works

To test my guardrails, I first used a standard dataset (HotpotQA split of the RAGbench dataset) to see how it handled complex questions. Subsequently, I developed a custom dataset comprising 50 question-answer pairs derived directly from current university study regulations.

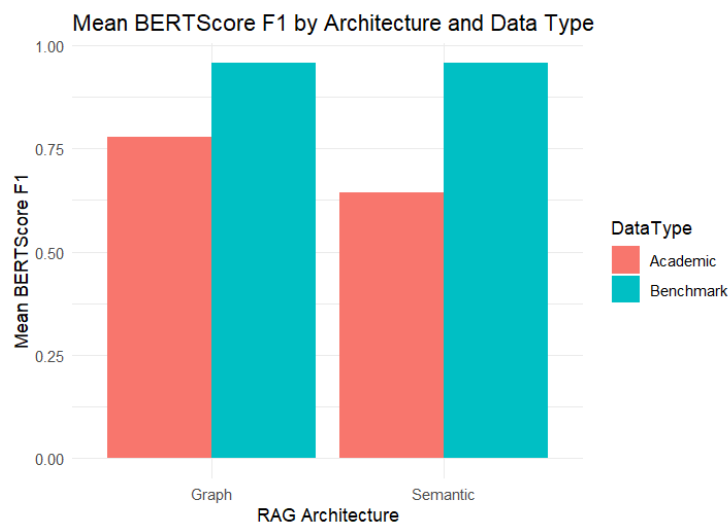
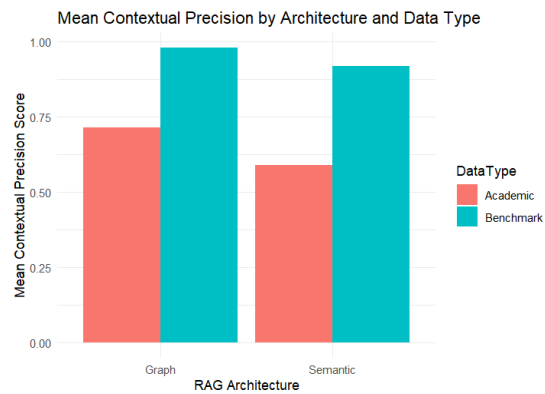
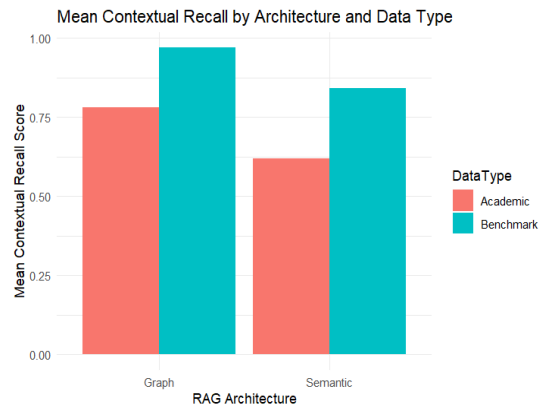
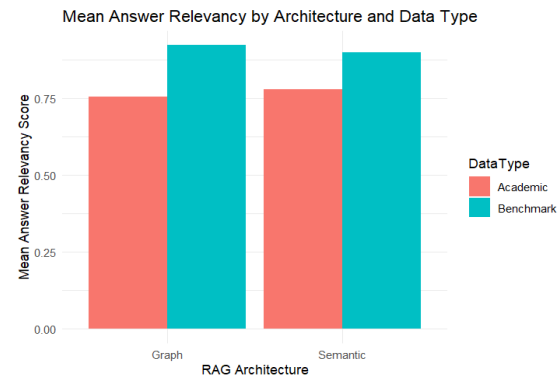
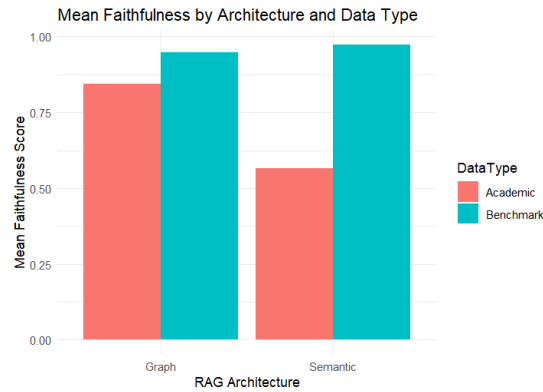
The results showed that the Knowledge Graph (GraphRAG) was a huge help. When looking at clean, standard test data, both systems did fine. But when I fed them the real, messy university PDFs, the standard RAG started failing and making things up. The GraphRAG, however, did a much better job of sticking to the facts and not hallucinating.

To test it out, I've already connected the semantic pipeline via RAGFlow chat assistant APIs to an external web interface, and it's working really well [\[Link\]](#).

The GraphRAG database, however, is currently running locally on my PC. To get it properly tested by other users in a real-world scenario, my next step could be to either transfer it to a Neo4j Aura cloud database or use a Docker image to deploy it on a local server.

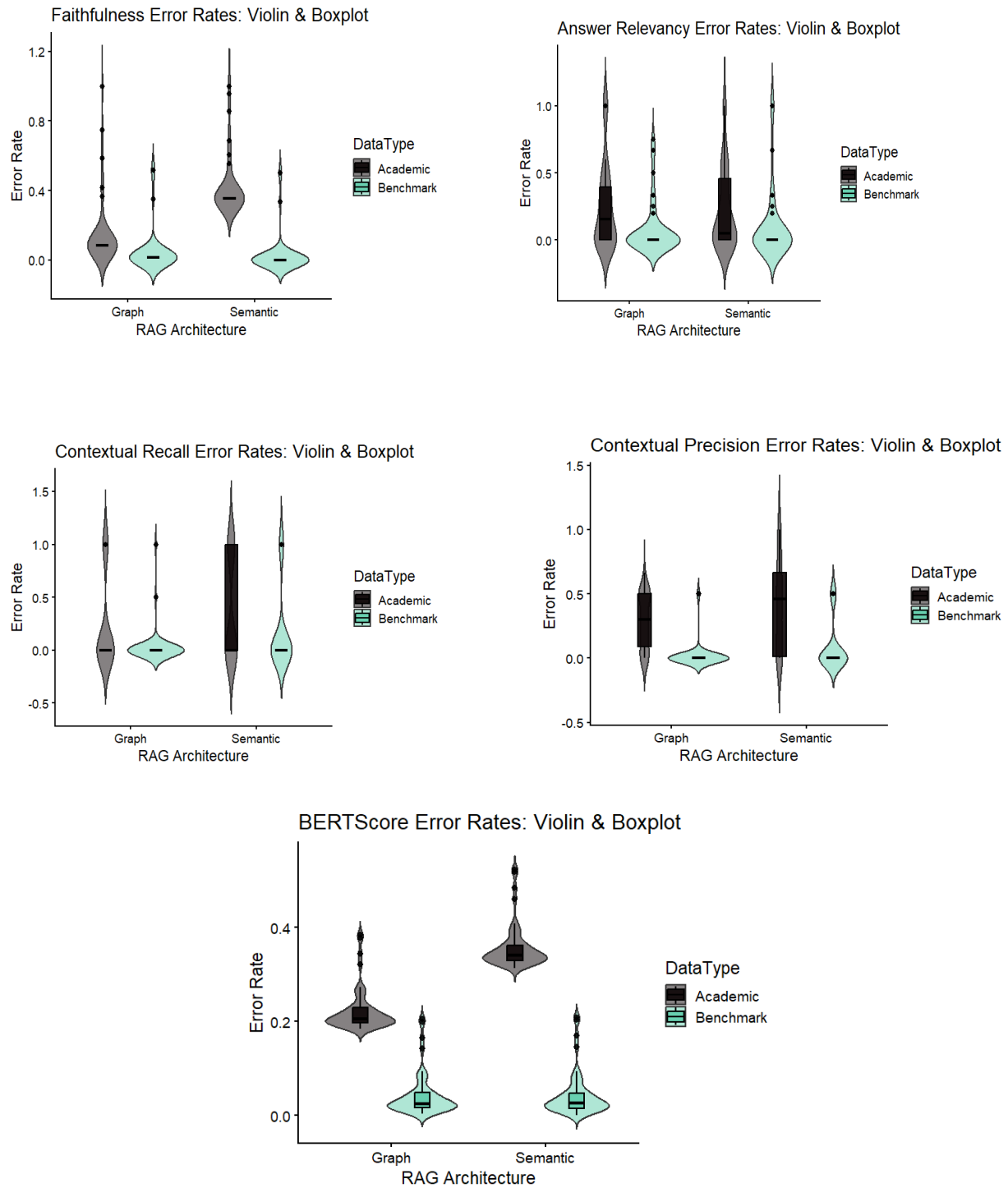
➤ Metrics Results & Plots

To rigorously evaluate the performance differences, I set up a 2x2 mixed factorial design (**Architecture** [Semantic vs. GraphRAG] * **Dataset** [RAGbench vs. Real PDFs]). To compare the results statistically, I used an ARC ANOVA (Aligned Rank Transform) using the ARTool package in R.



➤ Error Rate Visualizations

To further analyze the reliability and variance across the experimental conditions, the following error rate plots were generated from the ANOVA results.



2. What Worked Well

- **Graph search prevented hallucinations:** Because the Neo4j database links information together like a web, it was much better at handling complicated questions without inventing fake rules.
- **The multilingual model fixed everything:** Switching to the `BAAI/bge-m3` model completely solved the language barrier. English questions easily found the right German documents.

3. Where It Struggled

- **Building the graph took long:** While setting up a standard vector database (FAISS) is super-fast, building the Neo4j graph took a lot of time. The model had to scan every single word to find the "entities" (names, rules, dates) to map them out.
- **Messy data is still hard:** Even with all the guardrails, the performance dropped when moving from perfectly clean test data to real-world PDFs.

4. What I Would Do with More Time

- **Add a Re-ranker:** I would like to add a re-ranker model. It slows things down a bit, but it's great for filtering out bad search results, especially when trying to figure out if a rule is from the 2024 or 2025 rulebook.
- **Fine-tune the embeddings:** Instead of training a massive chat model, I'd fine-tune the smaller embedding model on my dataset.
- **Use a smarter judge:** Right now, my judge model and my chat model are the exact same (`gemma-4-31B`). If I had more computer power, I'd use a bigger, smarter model (like `Qwen 3.5 397B` or `Qwen 3.8 95B`) just to judge the answers, since bigger models and specially the Qwen series are better at reasoning.

5. Interesting Surprises

My biggest surprise was how badly my first embedding model failed. I started with a small, free model (`all-MiniLM-L6-v2`). Because I was asking questions in English, I thought it would be fine and it worked not back on the benchmark data which is entirely in English. But switching to regulation data, since the PDFs were 90% German, the model just started hallucinating completely fake answers because its an only-English model. As soon as I swapped it for a bigger, multilingual model (`BAAI/bge-m3`), the problem vanished. It was a great lesson in how the simplest parts of the pipeline can completely break your guardrails if you aren't paying attention.

Additionally, I mentioned that the graph-based retrieval pipeline was implemented utilizing the `SimpleKGPipeline` from the `neo4j_graphrag` library, configured to execute both semantic similarity searches and Cypher-based graph traversals. During the ingestion phase, the pipeline required the underlying model to perform Named Entity Recognition (NER) on every individual

data point and I had not expected that it will introduce a lot of latency during the database construction. Consequently, this computational overhead represents a notable trade-off when compared to the enhanced retrieval precision afforded by the knowledge graph architecture.